

COMPENDIO COMPLETO TEÓRICO-PRÁCTICO: TÉCNICAS DIGITALES II

Guía de Referencia Rápida, Registros RM0008, CMSIS y Ejercicios de Parcial (STM32F103)

UTN-FRC — Cátedra de Técnicas Digitales II (Año 2026)

Permitido para exámenes teórico/prácticos — Contiene referencias de registros de hardware STM32F103, mapa de memoria, estructuras CMSIS, cálculos de periféricos (SysTick, GPIO, EXTI, NVIC, TIM2 PWM, ADC1, USART1, SPI, I2C) y resoluciones modelo de parciales.

1. Arquitectura ARM Cortex-M3 y Mapa de Memoria (Bare Metal vs CMSIS)

1.1 Arquitectura del Núcleo ARM Cortex-M3

- **Arquitectura de Bus:** Harvard modificada (buses de instrucciones I-Code/D-Code y datos System independientes).
- **Pipeline:** 3 etapas (*Fetch, Decode, Execute*). Promedio de ejecución próximo a 1 CPI.
- **Registros del Procesador:**
 - **R0 - R12:** Registros de propósito general de 32 bits.
 - **R13 (SP - Stack Pointer):** Puntero de pila primario. Físicamente son 2 punteros:
 - **MSP (Main Stack Pointer):** Usado por defecto, en Reset y en manejadores de interrupción (Handler Mode).
 - **PSP (Process Stack Pointer):** Usado opcionalmente en tareas de usuario (Thread Mode) en Sistemas Operativos (RTOS).
 - **R14 (LR - Link Register):** Almacena la dirección de retorno de subrutinas y código de retorno de interrupción (EXC_RETURN).
 - **R15 (PC - Program Counter):** Dirección de la siguiente instrucción a ejecutar.
 - **xPSR (Program Status Register):** Combinación de APSR (Banderas *N, Z, C, V*), IPSR (Número de excepción activa) y EPSR (Estado Thumb, bit T siempre 1).
- **Modos de Operación y Privilegios:**
 - **Thread Mode:** Modo normal de ejecución de código. Puede ser Privilegiado o No Privilegiado.
 - **Handler Mode:** Modo activado automáticamente durante una excepción o interrupción. Siempre es Privilegiado.

1.2 Mapa de Memoria y Acceso Bare Metal Puro vs CMSIS

El mapa de memoria de 32 bits (4 GB) organiza las direcciones fijas del sistema:

- **Flash Memory:** 0x0800 0000 a 0x0801 FFFF (hasta 128 KB en STM32F103C8).
- **SRAM:** 0x2000 0000 a 0x2000 4FFF (20 KB).
- **Periféricos APB1 (hasta 36 MHz):** 0x4000 0000 (TIM2, TIM3, USART2, I2C1, SPI2).

- **Periféricos APB2 (hasta 72 MHz):** 0x4001 0000 (AFIO, EXTI, GPIOA..GPIOC, ADC1, TIM1, USART1, SPI1).
- **Periféricos AHB:** 0x4002 0000 (RCC, Flash Interface).
- **SCS / Core Peripherals (NVIC, SysTick):** 0xE000 E000.

Comparación de Acceso a Registros:

```

1 // --- 1. ACCESO BARE METAL DIRECTO (Punteros volatiles a direcciones fijas) ---
2 #define RCC_BASE      0x40021000UL
3 #define GPIOC_BASE     0x40011000UL
4 #define RCC_APB2ENR    (*(volatile uint32_t *) (RCC_BASE + 0x18))
5 #define GPIOC_CRH      (*(volatile uint32_t *) (GPIOC_BASE + 0x04))
6 #define GPIOC_ODR      (*(volatile uint32_t *) (GPIOC_BASE + 0x0C))
7
8 // --- 2. ACCESO CMSIS (Uso de punteros a estructuras definidas en stm32f1xx.h) ---
9 #include "stm32f1xx.h"
10 RCC->APB2ENR |= RCC_APB2ENR_IOPCEN; // Enciende reloj Puerto C
11 GPIOC->CRH &= ~GPIO_CRH_CNF13_Msk; // Modifica campos de bits
12 GPIOC->ODR ^= GPIO_ODR_ODR13;      // Conmuta bit PC13

```

Listing 1: Bare Metal Directo (Punteros) vs CMSIS (Estructuras de C)

2. GPIO (Puertos de Entrada/Salida General) y Sistema de Relojes (RCC)

2.1 Sistema de Relojes (RCC) y Prescalers

- **Fuentes de Reloj (SYSCLK):**
 - **HSI:** Oscilador interno de 8 MHz (RC de alta precisión). Reloj por defecto al reset.
 - **HSE:** Oscilador de cristal externo (8 MHz típico en BluePill).
 - **PLL:** Multiplicador de frecuencia interno. $SYSCLK = HSE \times 9 = 72 \text{ MHz}$.
- **Buses de Sistema y Frecuencias Máximas:**
 - **AHB (HCLK):** Reloj de CPU y memoria. Máximo 72 MHz.
 - **APB2 (PCLK2):** Bus rápido de periféricos (GPIO, ADC1, USART1, SPI1). Máximo 72 MHz.
 - **APB1 (PCLK1):** Bus lento de periféricos (TIM2..TIM4, USART2, I2C1, SPI2). Máximo 36 MHz.
- **Registro RCC->APB2ENR (Habilitación de Periféricos APB2):**
 - Bit 0 AFIOEN: Habilita reloj de funciones alternativas y EXTI.
 - Bit 2 IOPAEN: Habilita reloj del Puerto GPIOA.
 - Bit 3 IOPBEN: Habilita reloj del Puerto GPIOB.
 - Bit 4 IOPCEN: Habilita reloj del Puerto GPIOC.
 - Bit 9 ADC1EN: Habilita reloj del convertor ADC1.
 - Bit 14 USART1EN: Habilita reloj del puerto serie USART1.

2.2 Configuración de Registros GPIO (CRL y CRH)

Cada puerto GPIO (GPIOA, GPIOB, GPIOC) maneja 16 pines. Los pines se configuran mediante 2 registros de 32 bits (4 bits por pin):

- **GPIOx->CRL:** Configura los pines del 0 al 7.

- GPIOx->CRH: Configura los pines del 8 al 15.

Tabla 1: Valores del campo de 4 bits [CNF1:CNF0 MODE1:MODE0] para cada pin

Bits (CNF[1:0] MODE[1:0])	Valor Hex	Modo	Descripción / Uso Típico
00 00	0x0	Entrada Analógica	Para ADC (Desactiva buffers digitales)
01 00	0x4	Entrada Flotante	Entrada digital sin res. interna / UA
10 00	0x8	Entrada Pull-Up/Pull-Down	PULL-DOWN si ODR=0, PULL-UP si ODR=1
00 01	0x1	Salida Push-Pull (10 MHz)	Salida digital estándar velocidad media
00 10	0x2	Salida Push-Pull (2 MHz)	Salida digital estándar bajo consumo
00 11	0x3	Salida Push-Pull (50 MHz)	Salida digital estándar alta velocidad
10 11	0xB	Alt. Function Push-Pull (50 MHz)	Salida controlada por periférico (UART, I2C, etc.)

Registros de Datos y Manipulación Atómica:

- GPIOx->ODR: Registro de salida de datos (lectura/escritura).
- GPIOx->IDR: Registro de entrada de datos (solo lectura).
- GPIOx->BSRR: Bit Set/Reset atómico. Bits [15:0] ponen en 1 el pin; Bits [31:16] ponen en 0 el pin.
- GPIOx->BRR: Bit Reset atómico. Bits [15:0] ponen en 0 el pin.

3. Temporizador de Sistema SysTick (Manejo de Tiempos y Retardos)

3.1 Registros del SysTick y Fórmulas

El SysTick es un temporizador descendente de 24 bits integrado directamente en el núcleo Cortex-M3.

- SysTick->CTRL: Registro de Control y Estado.
 - Bit 0 ENABLE: Habilita el contador (1 = ON).
 - Bit 1 TICKINT: Habilita la interrupción `SysTick_Handler` (1 = Habilitado).
 - Bit 2 CLKSOURCE: Fuente de reloj (0 = AHB/8, 1 = AHB directo / HCLK).
 - Bit 16 COUNTFLAG: Se pone en 1 cuando el contador llega a 0.
- SysTick->LOAD: Registro de valor de recarga (24 bits, máx 0x00FF FFFF = 16,777,215).
- SysTick->VAL: Registro de valor actual (se limpia escribiendo cualquier valor).

Cálculo del Valor de Recarga (LOAD):

$$T_{\text{interrupt}} = \frac{\text{LOAD} + 1}{f_{\text{SysTick}}} \Rightarrow \text{LOAD} = (T_{\text{interrupt}} \times f_{\text{SysTick}}) - 1$$

Tabla 2: Valores típicos de LOAD según frecuencia de procesador y fuente

Frecuencia CPU (HCLK)	Fuente CLKSOURCE	Período Deseado (T)	Valor LOAD
72 MHz	HCLK directo (72 MHz)	1 ms (1 kHz)	$(0,001 \times 72 \times 10^6) - 1 = \mathbf{71999}$
72 MHz	HCLK / 8 (9 MHz)	1 ms (1 kHz)	$(0,001 \times 9 \times 10^6) - 1 = \mathbf{8999}$
72 MHz	HCLK directo (72 MHz)	100 μ s (10 kHz)	$(100 \times 10^{-6} \times 72 \times 10^6) - 1 = \mathbf{7199}$
8 MHz	HCLK / 8 (1 MHz)	1 ms (1 kHz)	$(0,001 \times 1 \times 10^6) - 1 = \mathbf{999}$

3.2 Código de Implementación Base de Tiempo 1ms y Delay

```
1 #include "stm32f1xx.h"
2
3 volatile uint32_t ms_ticks = 0;
4
5 void systick_init_ms(void) {
6     // Usando CMSIS estandar (Configura 1ms automaticamente con SystemCoreClock = 72MHz)
7     SysTick_Config(SystemCoreClock / 1000);
8 }
9
10 void SysTick_Handler(void) {
11     ms_ticks++; // Incrementa contador cada 1 ms
12 }
13
14 void delay_ms(uint32_t time_ms) {
15     uint32_t start_time = ms_ticks;
16     while ((ms_ticks - start_time) < time_ms); // Espera no bloqueante activa
17 }
```

Listing 2: Base de tiempos precisa y retardo no bloqueante con SysTick

4. Interrupciones Externas (EXTI) y Controlador NVIC

4.1 Conceptos Teóricos del NVIC y EXTI

- **NVIC (Nested Vectored Interrupt Controller):** Manejador de interrupciones integrado en el núcleo Cortex-M3. Soporta vectorización automática (sin necesidad de consultar flags por software para determinar el vector) y anidamiento basado en prioridades.
- **Mecanismos de Eficiencia del NVIC:**
 - **Tail-Chaining:** Si al finalizar una ISR hay otra interrupción pendiente de igual o mayor prioridad, el microcontrolador enlaza la ejecución directamente a la nueva ISR ****sin restaurar (pop) ni apilar (push)**** nuevamente los registros de la pila. Reduce la latencia de 24 ciclos a solo 6 ciclos de reloj.
 - **Late Arrival:** Si durante el apilamiento de registros (stacking) de una interrupción de baja prioridad llega una interrupción de mayor prioridad, el procesador atiende inmediatamente a la de mayor prioridad primero.
 - **Stacking Automático (Apilado de 8 Registros):** Al ingresar a una ISR, el hardware guarda automáticamente en la pila (MSP): R0, R1, R2, R3, R12, LR, PC, xPSR.
- **Mapeo de Líneas EXTI (AFIO):** Existen 16 líneas de interrupción externa (EXTI0 a EXTI15). Se seleccionan mediante los registros AFIO->EXTICR[0..3]:
 - EXTI_CR1[3:0] selecciona la fuente para EXTI0: 0000=PA0, 0001=PB0, 0010=PC0.
- **Limpieza de Bandera Pendiente (EXTI->PR):** La bandera de interrupción se limpia ****escribiendo un 1 lógico**** en el bit correspondiente del registro EXTI->PR.

4.2 Código Completo EXTI0 en PA0 (Boton Pull-Down) conmuta LED en PB13

```
1 #include "stm32f1xx.h"
2
3 void EXTI0_IRQHandler(void) {
4     if (EXTI->PR & EXTI_PR_PRO) {           // Verificar si fue EXTI0
5         EXTI->PR |= EXTI_PR_PRO;           // Limpiar flag ESCRIBIENDO UN 1
6         GPIOB->ODR ^= (1 << 13);           // Conmutar LED en PB13
7     }
8 }
```

```

9
10 int main(void) {
11     // 1. Relojes GPIOA, GPIOB y AFIO
12     RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPBEN | RCC_APB2ENR_AFIOEN;
13
14     // 2. PA0 como Entrada con Pull-Down (MODE=00, CNF=10 -> 0x8, ODR=0)
15     GPIOA->CRL &= ~(0xF << 0);
16     GPIOA->CRL |= (0x8 << 0);
17     GPIOA->ODR &= ~(1 << 0);           // ODR=0 -> Pull-Down activo
18
19     // 3. PB13 como Salida Push-Pull 2MHz
20     GPIOB->CRH &= ~(0xF << 20);
21     GPIOB->CRH |= (0x2 << 20);
22
23     // 4. Mapear EXTI0 a PA0 en AFIO (Bits 3:0 de EXTICR1 en 0000 = PA0)
24     AFIO->EXTICR[0] &= ~(0xF << 0);
25
26     // 5. Configurar EXTI0: Desenmascarar (IMR) y Flanco Ascendente (RTSR)
27     EXTI->IMR |= EXTI_IMR_MR0;
28     EXTI->RTSR |= EXTI_RTSTR_TR0;       // Flanco ascendente
29     EXTI->FTSR &= ~EXTI_FTSR_TR0;
30
31     // 6. Habilitar interrupcion en el NVIC
32     NVIC_SetPriority(EXTI0_IRQn, 2);
33     NVIC_EnableIRQ(EXTI0_IRQn);
34
35     while (1) {
36         // Bucle en reposo. Todo se maneja por interrupcion
37     }
38 }

```

Listing 3: Implementacion CMSIS de EXTI0 con Pull-Down en PA0

5. Temporizadores de Propósito General (TIMx) y Señales PWM

5.1 Principio de Funcionamiento del PWM por Hardware

Los temporizadores de propósito general (TIM2, TIM3, TIM4) disponen de 4 canales independientes de comparación de salida (*Output Compare*).

- **Fórmula de Frecuencia del PWM:**

$$f_{\text{PWM}} = \frac{f_{\text{TIM}}}{(\text{PSC} + 1) \times (\text{ARR} + 1)}$$

Donde $f_{\text{TIM}} = 72 \text{ MHz}$ para TIM2 cuando APB1 tiene prescalador /2.

- **Fórmula del Ciclo de Trabajo (Duty Cycle %):**

$$\text{Duty Cycle \%} = \frac{\text{CCR}_x}{\text{ARR} + 1} \times 100 \quad \Rightarrow \quad \text{CCR}_x = \frac{\text{Duty \%}}{100} \times (\text{ARR} + 1)$$

- **Modo PWM 1 (OCxM = 110):** La salida se mantiene en nivel ALTO mientras el contador TIMx->CNT sea menor que el registro de comparación TIMx->CCR3, y cambia a nivel BAJO cuando el contador supera a CCR3.

5.2 Código Completo de Configuración PWM 1kHz en TIM2_CH3 (PA2)

```

1 #include "stm32f1xx.h"
2
3 void pwm_init(void) {
4     // 1. Relojes GPIOA, AFIO y TIM2

```

```

5  RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_AFIOEN;
6  RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
7
8  // 2. PA2 (TIM2_CH3) como Function Alternate Push-Pull 50MHz (0xB)
9  GPIOA->CRL &= ~(0xF << 8);
10 GPIOA->CRL |= (0xB << 8);
11
12 // 3. Frecuencia PWM = 1 kHz: PSC = 71 (F_cnt = 1MHz), ARR = 999 (Periodo = 1ms)
13 TIM2->PSC = 71;
14 TIM2->ARR = 999;
15
16 // 4. Configurar Modo PWM 1 en Canal 3 (CCMR2) y Habilitar Preload
17 TIM2->CCMR2 &= ~(TIM_CCMR2_OC3M);
18 TIM2->CCMR2 |= (6 << TIM_CCMR2_OC3M_Pos) | TIM_CCMR2_OC3PE;
19
20 // 5. Habilitar salida Canal 3 (CCER) y encender Timer (CR1)
21 TIM2->CCER |= TIM_CCER_CC3E;
22 TIM2->CR1 |= TIM_CR1_ARPE | TIM_CR1_CEN;
23 TIM2->EGR |= TIM_EGR_UG;
24 }
25
26 void set_duty_percent(uint8_t duty) {
27     // CCR3 = (Duty * (ARR + 1)) / 100
28     TIM2->CCR3 = (duty * 1000) / 100;
29 }

```

Listing 4: Configuración CMSIS de PWM en PA2 a 1kHz con Duty Cycle de 10 %, 50 % y 90 %

6. Conversor Analógico-Digital (ADC1)

6.1 Características y Registros del ADC

El STM32F103 integra convertidores Analógico-Digitales de 12 bits por aproximaciones sucesivas (SAR).

- **Rango de Entrada y Resolución:** 0 V a 3,3 V. Resolución de $2^{12} = 4096$ niveles (0 a 4095).
- **Reloj del ADC (ADCCLK):** Derivado de APB2 (72 MHz). Máximo permitido 14 MHz. Se debe configurar el prescalador a /6 (12 MHz) en `RCC->CFGR`.
- **Registros Clave del ADC1:**
 - `ADC1->CR2`: Control 2. Bit `ADON` (1 = Encender / Iniciar), Bit `SWSTART` (1 = Iniciar conversión por software), Bit `TSVREFE` (1 = Habilitar sensor de temp. interno).
 - `ADC1->SMPR2`: Tiempo de muestreo por canal. 3 bits por canal. 111 = 239,5 ciclos (máxima precisión).
 - `ADC1->SQR3`: Secuencia regular 3. Bits `SQ1[4:0]` almacenan el número de canal de la 1ª conversión.
 - `ADC1->SR`: Estado. Bit `EOC` (1 = End of Conversion).
 - `ADC1->DR`: Registro de datos (12 bits alineados a la derecha por defecto).

6.2 Código Completo Inicialización y Lectura ADC1 Canal 2 (PA2)

```

1  #include "stm32f1xx.h"
2
3  void adc_init(void) {
4      // 1. Relojes GPIOA y ADC1
5      RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_ADC1EN;
6
7      // 2. Prescalador ADCCLK = APB2 / 6 = 72MHz / 6 = 12MHz
8      RCC->CFGR &= ~RCC_CFGR_ADCPRE;

```

```

9   RCC->CFGR |= RCC_CFGR_ADCPRE_DIV6;
10
11   // 3. PA2 como Entrada Analogica (MODE=00, CNF=00 -> 0x0)
12   GPIOA->CRL &= ~(0xF << 8);
13
14   // 4. Tiempo de muestreo Canal 2 a 55.5 ciclos (SMPR2)
15   ADC1->SMPR2 &= ~(0x7 << 6);
16   ADC1->SMPR2 |= (0x5 << 6);
17
18   // 5. Encender ADC1 (Encender requiere poner ADON en 1 dos veces o esperar t_STAB)
19   ADC1->CR2 |= ADC_CR2_ADON;
20 }
21
22 uint16_t adc_read(uint8_t canal) {
23     // Configurar canal en la 1ra conversion de la secuencia (SQR3)
24     ADC1->SQR3 &= ~ADC_SQR3_SQ1;
25     ADC1->SQR3 |= (canal & 0x1F);
26
27     // Iniciar conversion por software
28     ADC1->CR2 |= ADC_CR2_ADON | ADC_CR2_SWSTART;
29
30     // Esperar fin de conversion (polling flag EOC en SR)
31     while (!(ADC1->SR & ADC_SR_EOC));
32
33     // Retornar valor convertido (Al leer DR se limpia EOC automaticamente)
34     return (uint16_t)(ADC1->DR & 0xFFFF);
35 }

```

Listing 5: Inicialización y función de lectura bloqueante por polling del ADC1

7. Comunicación Serie USART (BaudRate, Polling e Interrupciones)

7.1 Cálculo del Registro de Baudios (USART_BRR)

La fórmula general para la tasa de baudios en la UART del STM32 es:

$$\text{Baudrate} = \frac{f_{\text{PCLK}}}{16 \times \text{USARTDIV}} \Rightarrow \text{USARTDIV} = \frac{f_{\text{PCLK}}}{16 \times \text{Baudrate}}$$

El registro USART1->BRR guarda USARTDIV en formato de punto fijo: ****[15:4] Mantisa (12 bits)**** y ****[3:0] Fracción (4 bits)**** (0,0 a 15/16).

Tabla 3: Valores exactos de BRR para frecuencias de 72 MHz y 8 MHz

Reloj Bus (f_{PCLK})	Baudrate	USARTDIV	Mantisa (Hex)	Fracción ($\times 16$)	Valor USART_BRR
72 MHz (PCLK2)	9600 bps	468,75	468 (0x1D4)	$0,75 \times 16 = 12$ (0xC)	0x1D4C
72 MHz (PCLK2)	19200 bps	234,375	234 (0xEA)	$0,375 \times 16 = 6$ (0x6)	0xEA6
72 MHz (PCLK2)	115200 bps	39,0625	39 (0x27)	$0,0625 \times 16 = 1$ (0x1)	0x271
8 MHz (PCLK2)	9600 bps	52,0833	52 (0x34)	$0,0833 \times 16 = 1,33 \approx 1$	0x0341

7.2 Código Completo USART1 por Polling e Interrupción

```

1  #include "stm32f1xx.h"
2
3  volatile char rx_dato = 0;
4  volatile uint8_t rx_flag = 0;
5
6  void USART1_IRQHandler(void) {
7      if (USART1->SR & USART_SR_RXNE) {
8          rx_dato = (char)(USART1->DR & 0xFF); // Al leer DR se limpia el flag RXNE
9          rx_flag = 1;

```

```

10 }
11 }
12
13 void usart1_init_19200(void) {
14     // 1. Relojes GPIOA, AFIO y USART1
15     RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_AFIOEN | RCC_APB2ENR_USART1EN;
16
17     // 2. PA9 (TX) -> Alternate Function Push-Pull 2MHz (0x9)
18     GPIOA->CRH &= ~(0xF << 4);
19     GPIOA->CRH |= (0x9 << 4);
20
21     // 3. PA10 (RX) -> Input Floating (0x4)
22     GPIOA->CRH &= ~(0xF << 8);
23     GPIOA->CRH |= (0x4 << 8);
24
25     // 4. Configurar 19200 bps a 72MHz (BRR = 0xEA6)
26     USART1->BRR = 0xEA6;
27
28     // 5. Habilitar TX, RX, USART1 e Interrupcion RXNEIE
29     USART1->CR1 |= USART_CR1_UE | USART_CR1_TE | USART_CR1_RE | USART_CR1_RXNEIE;
30
31     // 6. NVIC Enable
32     NVIC_SetPriority(USART1_IRQn, 1);
33     NVIC_EnableIRQ(USART1_IRQn);
34 }
35
36 void usart1_send_char(char c) {
37     while (!(USART1->SR & USART_SR_TXE)); // Esperar que el buffer de TX este vacio
38     USART1->DR = c;
39 }

```

Listing 6: Configuración USART1 19200bps / 9600bps 8N1 e interrupción RX

8. Protocolos de Comunicación Serie Síncrona: SPI e I2C (Guía Teórica)

8.1 Protocolo SPI (Serial Peripheral Interface)

- **Características:** Síncrono, Full-Duplex, Arquitectura Master-Slave.
- **Líneas de Señal:**
 - **MOSI (Master Out Slave In):** Transmisión del Maestro al Esclavo.
 - **MISO (Master In Slave Out):** Transmisión del Esclavo al Maestro.
 - **SCK (Serial Clock):** Reloj de sincronismo generado exclusivamente por el Maestro.
 - **NSS / CS (Chip Select):** Selección de esclavo (activo en bajo).
- **Modos de Operación (Polaridad CPOL y Fase CPHA):**
 - **CPOL = 0:** Reloj en estado BAJO en reposo. **CPOL = 1:** Reloj en ALTO en reposo.
 - **CPHA = 0:** Muestreo en el primer flanco del reloj. **CPHA = 1:** Muestreo en el segundo flanco.
- **Registros STM32:** SPIx->CR1 (SPE=Enable, MSTR=Master, BR=Prescaler, CPOL, CPHA), SPIx->DR, SPIx->SR (TXE, RXNE, BSY).

8.2 Protocolo I2C (Inter-Integrated Circuit)

- **Características:** Síncrono, Half-Duplex, Bus de 2 hilos, Multi-Master / Multi-Slave.
- **Líneas de Señal (Configuración Open-Drain con Pull-Up externa a 3.3V):**
 - **SDA (Serial Data):** Línea bidireccional de datos.

- **SCL (Serial Clock):** Línea de reloj.

■ **Condiciones de Enmarcado:**

- **START (S):** Transición de ALTO a BAJO en SDA mientras SCL permanece en ALTO.
- **STOP (P):** Transición de BAJO a ALTO en SDA mientras SCL permanece en ALTO.
- **ACK / NACK:** El receptor pone SDA en BAJO durante el noveno pulso de reloj para confirmar la recepción (ACK).

- **Direccionamiento:** 7 bits de dirección + 1 bit R/W (0 = Escritura, 1 = Lectura).

- **Registros STM32:** I2Cx->CR1 (PE=Enable, START, STOP, ACK), I2Cx->OAR1 (Dirección propia), I2Cx->DR, I2Cx->SR1 (SB, ADDR, TXE, RXNE, BTF).

9. Tabla de Referencia Rápida para el Examen Práctico

Tabla 4: Cheat-Sheet de Respuestas Rápidas para Exámenes

Periférico / Tarea	Registros Envolventes	Valores Típicos / Claves
GPIO PC13 Salida 2MHz	GPIOC->CRH	Bits [23:20] = 0x2 (MODE=10, CNF=00)
GPIO PA0 Entrada Pull-Down	GPIOA->CRL, ODR	Bits [3:0] = 0x8 (MODE=00, CNF=10), ODR0 = 0
GPIO PA2 Alternate Func. 50MHz	GPIOA->CRL	Bits [11:8] = 0xB (MODE=11, CNF=10)
SysTick 1ms @ 72MHz	SysTick->LOAD, CTRL	LOAD = 71999, CTRL = 0x7 (Enable+Int+ClkSrc)
SysTick 1ms CMSIS	SysTick_Config(...)	SysTick_Config(SystemCoreClock / 1000)
EXTIO en PA0	AFIO->EXTICR[0], IMR, RTSR	EXTICR1=0x0, IMR =1, RTSR =1, Limpiar: PR =1
NVIC Habilitación EXTIO	NVIC_EnableIRQ(EXTIO_IRQn)	Directo: NVIC_ISE0 = (1 << 6)
ADC1 Canal 2 PA2	ADC1->CR2, SMPR2, SQR3	SMPR2 = (5 << 6), SQR3=2, Polling EOC
TIM2 PWM 1kHz @ 72MHz	TIM2->PSC, ARR, CCMR2, CCER	PSC=71, ARR=999, CCMR2 =0x60, CCER =1, CCR3=Duty
USART1 19200bps @ 72MHz	USART1->BRR, CR1	BRR = 0xEA6, CR1 = UE TE RE RXNEIE
USART1 9600bps @ 72MHz	USART1->BRR	BRR = 0x1D4C